



Air University, Islamabad

**Software Requirement Specification
(SRS DOCUMENT)**

for

SmartRent

Version 1.0

By

Asad Baig 221811

Zoha Javed 221848

Urwa Zamir 222687

Supervisor

Sir Zain Abubaker

Bachelor of Science in Software Engineering (2022-2026)

Revision History

Name	Date	Reason for Changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
Sir Zain Abubaker

Signature_____

Table of Contents

1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Modules.....	2
1.4 Overview	4
2. Overall Description	6
2.1 Product Perspective	6
2.2 User classes and characteristics	7
2.3 Operating Environment	8
2.4 Design and Implementation Constraints.....	9
3. Requirement Identifying Technique.....	10
4. Functional Requirements.....	12
5. Non-Functional Requirements	18
5.1 Reliability	18
5.2 Usability	18
5.3 Performance.....	19
5.4 Security	19
6. External Interface Requirements.....	20
6.1 User Interfaces Requirements.....	20
6.2 Software interfaces	20
6.3 Hardware interfaces	21
6.4 Communications interfaces	21
7. References	22

1. Introduction

SmartRent is a blockchain- and AI-powered web application designed to streamline and secure rental property transactions in Pakistan. It replaces outdated, paper-based processes with a transparent, fraud-resistant digital platform tailored for landlords and tenants. Key features include secure contract creation, manual user verification, AI-driven insights, and automated rent management. This Software Requirements Specification (SRS) outlines the system's functional and non-functional requirements, providing a clear understanding for all stakeholders involved in its development and implementation.

1.1 Purpose

The purpose of this SRS is to define the software requirements for **SmartRent**, a secure and intelligent rental platform that simplifies and safeguards property rental processes in Pakistan. It documents the intended behavior, features, and constraints of the system. The primary audience includes:

- **Developers:** to understand system architecture and implement features accordingly.
- **Testers:** to derive test cases from the documented requirements.
- **Project Managers:** to plan and track project progress.
- **Stakeholders:** such as landlords, tenants, and real estate agents to align expectations.
- **Documentation Writers:** to create user manuals and support content.

1.2 Scope

SmartRent is a blockchain-powered rental management platform designed to streamline and secure rental transactions in Pakistan's real estate market. The software aims to eliminate common issues such as fraud, legal ambiguity, and inefficiencies in communication between landlords and tenants. By offering a transparent, digital solution, SmartRent supports the overarching goal of making rental processes more trustworthy, legally compliant, and user-friendly.

The platform is intended for use by landlords, tenants, and property managers who require a reliable digital solution to manage rental agreements, payments, and disputes without relying on traditional, paper-based processes. It contributes to project goals by improving transparency, automating key rental operations, and ensuring compliance with Pakistan's rental laws.

Key Features and Functions:

- **Blockchain-Based Contract Management:** Create, store, and access secure, immutable rental agreements.
- **Manual Identity Verification:** Authenticate users before login through document-based verification by administrators.
- **Legal Compliance Assistance:** Automatically detect violations of rental regulations and suggest compliant alternatives.
- **AI-Powered Rental Pricing:** Recommend fair rental prices using historical data, market trends, and property attributes.

- **Integrated Payment System:** Secure rent collection through multiple Pakistani banking channels.
- **Automated Notifications and Reminders:** Keep stakeholders informed of due dates, payments, and contract updates.
- **Dispute Resolution System:** Facilitate unbiased conflict resolution with blockchain-recorded communication and evidence.
- **User-Friendly Interface:** Ensure accessibility across devices and varying internet conditions in Pakistan.

SmartRent does **not** include physical property management services such as maintenance or inspections. Instead, its focus is on enhancing digital rental interactions through secure, legally sound, and transparent technology.

1.3 Modules

Contract Creation

- Generate legally binding tenant agreements with step-by-step form assistance
- Include standard terms covering deposits, repairs, and termination policies
- Allow customization of special terms and conditions
- Provide template options for different property types (apartment, house, commercial)
- Store agreements on blockchain with timestamp and digital signatures
- Enable download/print of finalized contracts in multiple formats (PDF, DOC)

Contract Management

- Validate contract details against local rental laws automatically
- Flag non-compliant or unusual terms for review
- Provide intuitive dashboard for tracking active/expired contracts
- Enable contract renewals with updated terms
- Allow amendments with change tracking
- Set automated reminders for contract expiration dates
- Generate contract analytics (average duration, common clauses)

Manual Identity Verification

- Allow users to manually upload identity documents (e.g., CNIC, passport, driver's license) during registration
- Provide a manual verification process by admin before account activation
- Track verification status (Pending, Verified, Rejected) with user notifications
- Securely store uploaded documents with encryption

- Allow limited temporary access for users pending verification
- Maintain full verification history and audit logs for security compliance

Mutual Reviewing System

- Display comprehensive rental history for both parties
- Implement weighted rating system (timeliness, cleanliness, communication)
- Allow photo/video evidence in reviews
- Provide response mechanism for reviewed parties
- Calculate and display trust scores
- Maintain review integrity through verification checks

AI Pricing Suggestions

- Analyze market trends and comparable properties
- Consider neighborhood facilities and infrastructure
- Adjust for property condition and features
- Provide pricing range with Security indicators
- Show historical pricing trends for area
- Allow manual override with justification
- Learn from user adjustments to improve suggestions

AI Budget-Based Location Suggestions

- Filter properties by customizable budget ranges
- Highlight best value options in preferred areas
- Show travel times of the user's selected places
- Overlay neighborhood safety data
- Indicate price trends in suggested areas
- Allow saving of preferred search parameters
- Provide alternative area suggestions when budget is limiting

Document Management

- Secure cloud storage for all rental documents
- Organize documents by category and property
- Set access permissions for different parties
- Enable document sharing with expiration dates
- Provide digital signature capability
- Track document views and downloads

- Integrate with contract management system

Automated Payments

- Support multiple payment methods (bank transfer, cards, mobile wallets)
- Schedule recurring rent payments
- Generate automatic receipts and invoices
- Handle partial payments and payment plans
- Provide payment dispute resolution process
- Track payment history with search functionality
- Integrate with accounting software

Notifications and Reminders

- Customizable alert preferences (email, SMS, app notifications)
- Automated rent due reminders with payment links
- Inspection and maintenance alerts
- Contract milestone notifications
- Important deadline warnings
- System updates and announcements
- Emergency communication channel

Dispute Resolution

- Structured dispute submission forms
- Evidence upload and organization tools
- Dispute resolution request system
- Case tracking and status updates
- Resolution documentation generation
- Third-party arbitrator integration option

1.4 Overview

This SRS document provides a complete description of the **SmartRent** system.

It starts with the **Introduction** section, which includes:

- The purpose of the system.
- The overall scope of the project.
- Key modules and features.

The **Overall Description** section explains:

- The background and context of the product.
- The types of users.
- The operating environment and constraints.

The **Requirement Identifying Technique** section describes:

- How the system requirements were gathered (e.g., use cases).

The **Functional Requirements** section lists:

- All the main features like login, contract creation, manual identity verification, AI suggestions, and payment processing.

The **Non-Functional Requirements** section defines:

- Qualities such as reliability, security, performance, and usability.

The **External Interface Requirements** section explains:

- How the system will interact with users, external software, hardware, and communication tools.

The **References** section includes:

- All sources and documents referred to during the project.

The SRS is structured to help:

- Developers understand what to build.
- Testers know what to test.
- Managers track progress.
- Users and stakeholders understand the system goals.

2. Overall Description

This section gives a general overview of the SmartRent system, where and how it will be used, who will use it, and what limitations or assumptions the team must consider during development

2.1 Product Perspective

SmartRent is a completely **new web-based application**, not a part of any existing system.

SmartRent is an **entirely new product**, designed from the ground up to address major problems in the Pakistani rental property market. It is not a continuation, upgrade, or replacement of any existing software system. It was designed from scratch to solve current problems in Pakistan's rental market, such as fraud, fake listings, and lack of trust.

Unlike existing property platforms (such as Zameen.com or Graana.com) that only allow property listings, SmartRent provides a **complete end-to-end solution** for the rental process. It includes:

- **Blockchain-based contract creation and storage**, ensuring legal security and transparency.
- **Manual identity verification** by platform admins to confirm the legitimacy of users.
- **AI-powered rental price and location suggestions** based on market data.
- **Online rent payments** with history tracking and receipts.
- **Document storage** and a **built-in dispute resolution system** to handle any rental-related issues.

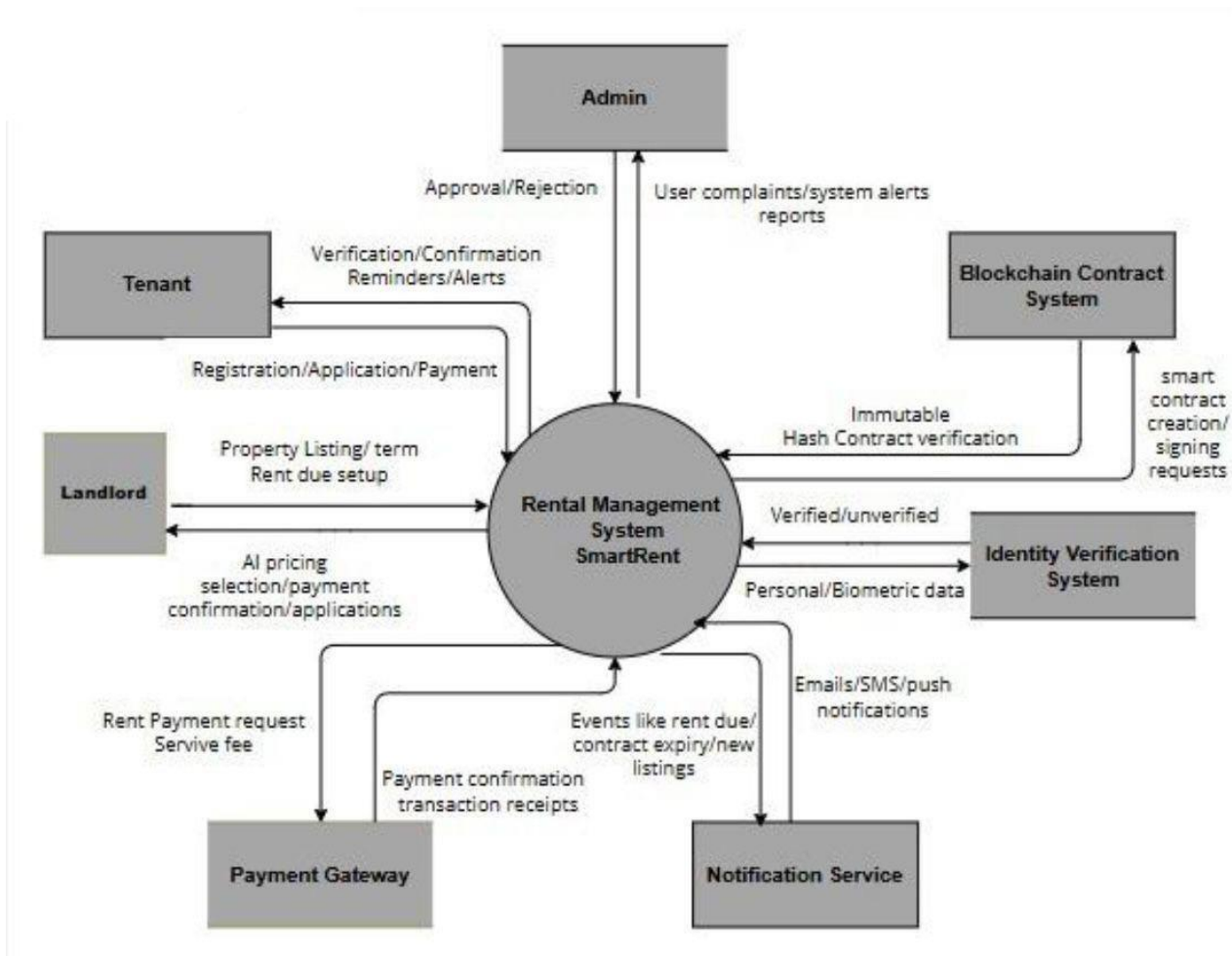
SmartRent is a **standalone web application** but may interact with:

- Local **payment gateways** (e.g., JazzCash, Easypaisa, bank APIs),
- **Cloud storage services** for document handling,
- In the future, possible integration with **government APIs** for identity verification.

The product is initially aimed at users in **Islamabad**, with scalability options for deployment in other regions.

Figure 1: SmartRent Context Diagram

The context diagram below illustrates the interaction between external entities (Tenant, Landlord Admin, etc.) and the SmartRent system.



2.2 User classes and characteristics

The following are the main user classes expected to interact with the **SmartRent** system:

User Class	Description and Characteristics
Tenant	- A user looking to rent a property. - Can search listings, submit rental applications, and sign contracts. - Required to upload identification documents for manual verification. - Can make online rent payments and leave reviews for landlords.
Landlord	- A property owner listing one or more rental properties. - Can create and manage rental contracts, receive rent payments, and view tenant reviews. - Must be verified by admin before listing properties. - Interested in secure transactions and tenant credibility.
Admin	- Responsible for verifying user identities by checking uploaded documents. - Manages user account approvals and rejects suspicious profiles. - Handles dispute resolution and monitors system activity. - Has full access to all user records and logs.

Visitor (Unregistered User)	- Can browse available properties without logging in. - Must register to access core features like applying for a property, creating listings, or signing contracts. - Can become a tenant or landlord upon registration and verification.
------------------------------------	--

Each user type has specific access rights and functions designed to ensure security, trust, and ease of use on the SmartRent platform.

2.3 Operating Environment

The **SmartRent** application will operate in a web-based environment, accessible through various browsers and devices. The key details of its operating environment are listed below:

OE-1: *The system shall operate correctly with the following modern web browsers:*

- Google Chrome (latest versions)
- Mozilla Firefox (latest versions)
- Microsoft Edge
- Apple Safari (macOS and iOS)

OE-2: *The system shall be compatible with the following operating systems:*

- Windows 10 and above
- macOS
- Android (version 8 and above)
- iOS (version 12 and above)

OE-3: *The system shall support responsive design to ensure usability on various screen sizes including desktops, laptops, tablets, and smartphones.*

OE-4: *The application backend and database will be hosted on a **cloud platform** such as **Heroku**, **AWS**, or **MongoDB Atlas**, ensuring high availability and scalability.*

OE-5: *The geographical focus of the system will initially be limited to **Islamabad, Pakistan**, with future expansion planned for other cities.*

OE-6: *All databases and sensitive user data shall be stored in encrypted form in **secure cloud storage** hosted in compliance with **data privacy regulations** applicable in Pakistan.*

OE-7: *Internet connectivity is required for all functionalities of the system, including contract creation, document uploads, and online payments.*

2.4 Design and Implementation Constraints

The design and development of the **SmartRent** system must follow certain technical constraints due to technology choices, platform limitations, and project requirements. These constraints are listed below:

CON-1: *The system shall be developed using the **MERN Stack**, which includes:*

- **MongoDB** as the database,
- **Express.js** for server-side development,
- **React.js** for the front-end interface, and
- **Node.js** as the runtime environment.

CON-2: *The system shall use **blockchain technology** to store and secure rental contracts, ensuring transparency and tamper-proof records.*

CON-3: *User **identity verification** shall be performed **manually by the admin**, the admin will check and verify the CNIC or document then move to the contract creation.*

CON-4: *The system shall only support **local payment gateways** (e.g., JazzCash, Easypaisa, bank APIs) for secure and accessible online rent transactions.*

CON-5: *All data must be encrypted during **storage and transmission**, following data privacy regulations in Pakistan.*

CON-6: *The application must be **web-based and responsive**, with no native mobile application required during the initial release.*

CON-7: *The system must comply with **local rental laws and data protection policies** applicable in Pakistan, especially in Islamabad during the initial phase.*

3. Requirement Identifying Technique

To identify the functional and non-functional requirements of the **SmartRent** system, the most suitable technique used was the **Use Case Methodology**. Since SmartRent is an **interactive end-user web application**, involving multiple user roles such as tenants, landlords, and admins, use cases provide a clear and structured way to understand user interactions with the system.

The following approaches were applied:

- **Use Case Diagrams** were developed to visually represent the interaction between users (actors) and the system's core functionalities (use cases). This helped define the boundaries of the system and the responsibilities of each user class.
- **Detailed Use Cases** were written for key system features such as:
 - User registration and login
 - Rental agreement creation
 - Manual user verification by admin
 - AI-based price recommendations
 - Rent payment processing
 - Review and rating system
 - Dispute resolution

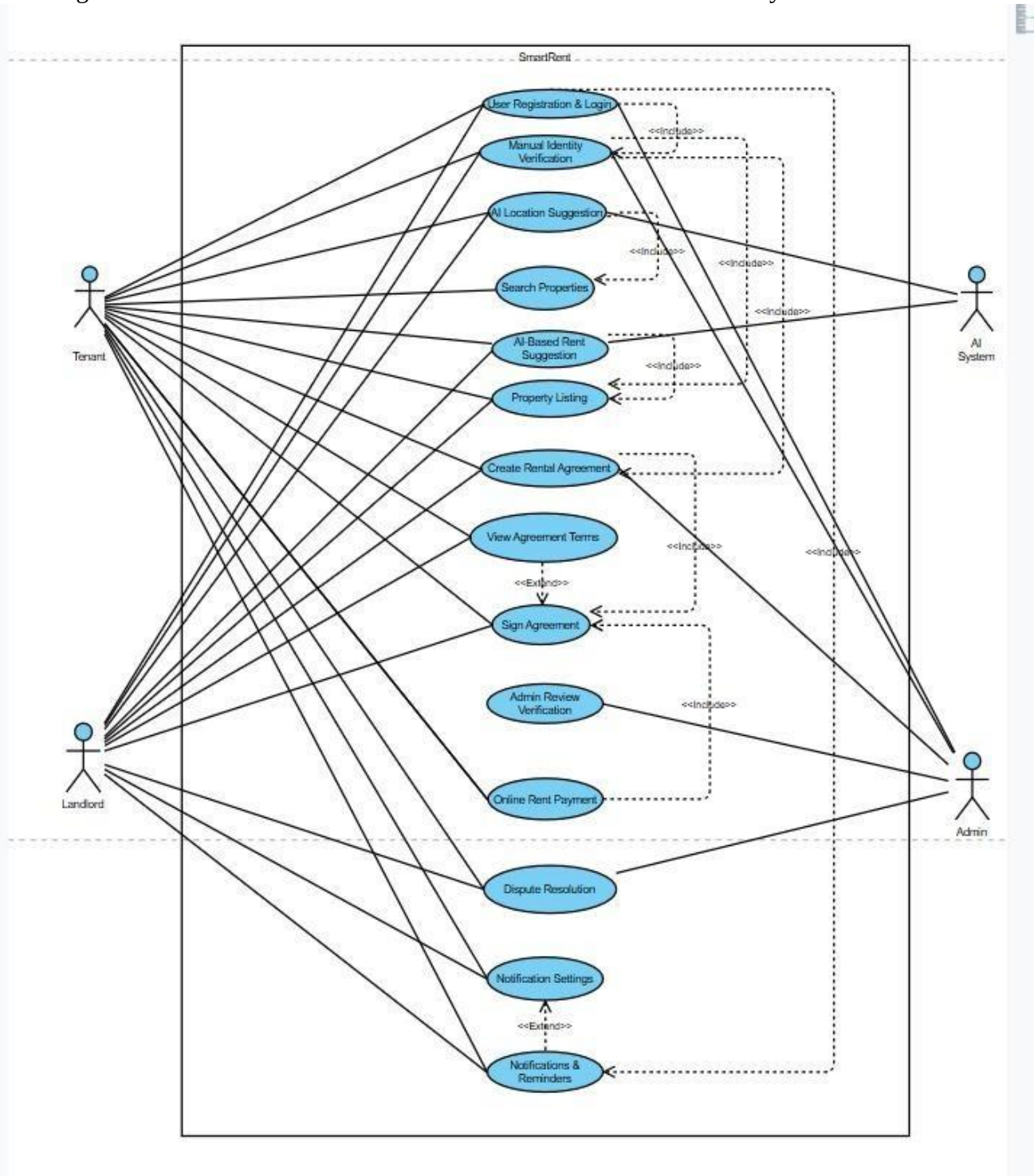
These detailed use cases describe:

- Actor roles and goals
- Preconditions and postconditions
- Normal and alternate flow of events
- Exceptions and system responses

This technique provided a comprehensive understanding of the system's behavior and helped in accurately defining the functional requirements.

Figure 2: SmartRent Use Case Diagram

This diagram shows the actors involved and their interactions with various system use cases.



4. Functional Requirements

This section outlines the detailed functional requirements of the **SmartRent** application. Each requirement describes a key capability of the system, written from the perspective of either the user or the system, and aligned with the use case approach.

4.1 Functional Requirement FR-1

Table 1: Description of FR-1

Identifier	FR-1
Title	User Registration and Login
Requirement	The user (tenant or landlord) shall be able to create an account using a valid email address and password. After registration, the system shall allow secure login with credentials and provide access to user-specific features based on their role.
Source	Use Case: User Registration and Authentication
Rationale	To enable users to securely access the platform and its features through personal accounts.
Business Rule	- Email address must be unique per user. - Password must meet minimum strength requirements (e.g., at least 8 characters, 1 symbol).
Dependencies	None
Priority	High

4.2 Functional Requirement FR-2

Table 2: Description of FR-2

Identifier	FR-2
Title	Manual Identity Verification
Requirement	After registration, the user shall upload a valid government-issued ID (e.g., CNIC). The system shall forward the document to the admin panel for review. The admin shall accept or reject the verification request based on document authenticity. Verified status must be shown in the user profile.
Source	Use Case:Manual Identity Verification
Rationale	To ensure all users are real and trustworthy before they can create or sign rental agreements.
Business Rule	- Only verified users may list properties or sign agreements. - Admins must review and decide within 48 hours.
Dependencies	FR-1
Priority	High

4.3 Functional Requirement FR-3

Table 3: Description of FR-3

Identifier	FR-3
Title	Rental Agreement Creation and Management
Requirement	A verified landlord shall be able to create a digital rental agreement by entering details such as property address, rent amount, duration, and terms. The system shall validate inputs and store the finalized agreement securely on the blockchain. Both tenant and landlord shall receive a confirmation.
Source	Use Case: Create Rental Agreement
Rationale	To create tamper-proof, legally binding contracts that ensure transparency and security.
Business Rule	- Contracts must follow Pakistan's rental law. - Contract edits are allowed only before tenant approval.
Dependencies	FR-2

Priority	High
-----------------	------

4.4 Functional Requirement FR-4

Table 4: Description of FR-4

Identifier	FR-4
Title	AI-Based Rent Price Suggestion
Requirement	The system shall provide landlords with AI-generated rent price suggestions based on location, property type, size, and market data. The landlord may choose to accept or modify the suggested rent before finalizing the agreement.
Source	Use Case: AI-Based Pricing Suggestions
Rationale	To help landlords set fair and competitive prices while increasing tenant trust.
Business Rule	- AI model must be trained on up-to-date local market data. - Suggestions must include a comparison to average market prices.
Dependencies	FR-3
Priority	Medium

4.5 Functional Requirement FR-5

Table 5: Description of FR-5

Identifier	FR-5
Title	AI Budget-Based Location Suggestions
Requirement	Tenants shall input a monthly budget and preferred city; the system shall suggest neighbourhoods and listings that fit the budget, showing commute times and safety scores.
Source	Use Case: AI Budget-Based Location Suggestions
Rationale	Helps tenants find affordable areas quickly, improving platform usefulness.
Business Rule	• Suggestions limited to cities where market data coverage $\geq 80\%$ • Results must include at least three neighbourhoods or display “Insufficient data.”
Dependencies	FR-1 (Login), relies on external market-data feed
Priority	Medium

4.6 Functional Requirement FR-6

Table 6: Description of FR-6

Identifier	FR-6
Title	Online Rent Payment
Requirement	The system shall allow tenants to pay rent online using integrated local payment gateways (e.g., JazzCash, Easypaisa, or bank transfers). Upon successful payment, the system shall generate a receipt and update the payment history for both the tenant and landlord.
Source	Use Case: Payments
Rationale	To enable fast, secure, and traceable rent transactions without cash dependency.
Business Rule	- Payments must be recorded in PKR. - Receipts must be generated and emailed to both parties.
Dependencies	FR-3
Priority	High

4.7 Functional Requirement FR-7

Table 7: Description of FR-7

Identifier	FR-7
Title	Dispute Resolution Mechanism
Requirement	The system shall allow users to raise disputes with a reason and supporting documents. Admins shall review the case and respond with a resolution. System must log all dispute actions.
Source	Use Case: Transparent Dispute Resolution
Rationale	To provide a fair way to handle conflicts between landlords and tenants.
Business Rule	- Evidence files must be in PDF or image format. - Admin must respond within 72 hours.
Dependencies	FR-3
Priority	High

4.8 Functional Requirement FR-8

Table 8: Description of FR-8

Identifier	FR-8
Title	Secure Document Upload & Management
Requirement	The system shall let users upload rental-related documents (property deed, CNIC scan, utility bill, lease addenda) in PDF, JPG, or PNG format. Uploaded files shall be stored in encrypted cloud storage and be view-only for the counter-party.
Source	Use Case: Manage Rental Documents
Rationale	Ensures all legal and identity proofs are centrally available and tamper-proof.
Business Rule	• File size ≤ 10 MB each. • Only verified users (FR-2) can upload or view documents. • Deletion is disabled after a contract is signed.
Dependencies	FR-2 (Manual Identity Verification)
Priority	High

4.9 Functional Requirement FR-9

Table 9: Description of FR-9

Identifier	FR-9
Title	Notifications & Automated Reminders
Requirement	The system shall send email/SMS/app notifications for upcoming rent due dates, contract expiry, verification decisions, and unread dispute messages. Users may configure preferred channels.
Source	Module 9: Notifications and Reminders
Rationale	Keeps users informed, reducing late payments and missed deadlines.
Business Rule	• Rent reminders sent 3 days and 1 day before due date • Contract-expiry reminder sent 7 days in advance. • Users can opt-out of marketing messages but not critical alerts.
Dependencies	FR-1, FR-5, FR-7
Priority	Medium

4.10 Functional Requirement FR-10

Table 10: Description of FR-10

Identifier	FR-10
Title	Property Listing & Search
Requirement	Verified landlords shall be able to list properties with photos, rent, and amenities. Tenants shall search/filter listings by city, price range, property type, and availability dates.
Source	Implied in Modules 1 & 6; tenant/landlord core workflow
Rationale	Provides the essential marketplace features around which contracts and payments revolve.
Business Rule	• Listing becomes public only after landlord verification (FR-2). • Each photo ≤ 5 MB; max 10 photos per listing. • Search results paginated (20 per page).
Dependencies	FR-2, FR-8
Priority	High

5. Non-Functional Requirements

This section defines the non-functional (quality) requirements of the **SmartRent** system. These are critical for ensuring reliability, usability, and performance throughout the system's lifecycle. The requirements are clearly defined, measurable, and can be verified during testing.

5.1 Reliability

REL-1: The system shall maintain a **Mean Time Between Failures (MTBF)** of at least **500 hours** during regular operations.

REL-2: If something goes wrong (e.g., during contract creation or payment), the system should show a **clear error message** and let the user **try** again without losing their data.

REL-3: The system should **automatically back up** all important data (like contracts, user accounts, and payments) every day to prevent **data loss**.

REL-4: If the system fails, it should recover and become usable again within **2 minutes**.

.

5.2 Usability

USE-1: SmartRent should have a **user-friendly and intuitive interface**, so that landlords and tenants can perform their tasks without training.

USE-2: Users should be able to finish important tasks in **3 or fewer clicks/steps**.

USE-3: If users enter wrong data, the system should display **helpful error messages** and **real-time guidance**.

USE-4: The design should be **responsive**, adapting to **desktops, tablets, and smartphones**.

5.3 Performance

PERF-1: **95% of operations** (e.g., creating a contract or opening a document) should complete in **under 3 seconds** on a **10 Mbps** internet connection.

PERF-2: The system must handle at least **10,000 concurrent users** without slowing down.

PERF-3: Background processes like **user verification and payment** should complete in **under 5 seconds** even during **peak times**.

5.4 Security

SEC-1: The system must use **strong encryption** (e.g., **TLS 1.2+** and **AES-256**) to protect **personal data** (like **CNIC, contracts**).

SEC-2: **Multi-Factor Authentication (MFA)** is **mandatory for admin** and **optional for regular users**.

SEC-3: The system should maintain **audit logs of all user activities** (e.g., **contract creation, updates**).

SEC-4: If a user **fails to log in 5 times in 10 minutes**, the account should be **locked temporarily**.

5.5 Maintainability

MAINT-1: Code should follow **modular design** for easier updates and debugging.

MAINT-2: Code must be **well-documented and version-controlled** for easier handover and future development.

MAINT-3: All errors should be recorded in **detailed system logs** for faster debugging.

5.6 Portability and Compatibility

PORT-1: The system shall be compatible with **all major modern browsers**, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.

PORT-2: The system shall operate correctly on commonly used devices, including **desktops, laptops, tablets, and smartphones** with Android or iOS operating systems.

PORT-3: The system shall allow integration with **third-party services** such as local **payment gateways** (e.g., JazzCash, Easypaisa) and, optionally in future versions.

5.7 Privacy and Legal Compliance

COMP-1: The system shall **comply with all applicable laws and regulations** related to digital contracts, tenant-landlord agreements, and data protection in Pakistan, especially in **Islamabad**.

PRIV-1: The system shall follow **data minimization principles**, collecting only the minimum required user data necessary for performing essential functions.

PRIV-2: The system shall present users with **explicit consent options** before collecting, storing, or sharing personal data, particularly when using third-party services.

5.8 Support and Help

SUP-1: The system shall include a **technical support section**, such as a helpdesk or contact form, allowing users to raise support tickets or request assistance.

SUP-2: The system shall include a **Frequently Asked Questions (FAQ)** page and a downloadable **User Manual** to help users understand system functionality without external support.

.

6. External Interface Requirements

This section describes how the **SmartRent** system will interact with its users and external software, hardware, and communication systems. It ensures that the system connects and communicates effectively with all necessary components.

6.1 User Interfaces Requirements

The SmartRent system will have a **responsive, web-based graphical user interface (GUI)** accessible through modern web browsers on both desktop and mobile devices. The design will be consistent across all modules and follow common UI/UX standards for usability.

The following characteristics will be applied:

- The interface will follow a **clean and modern style guide**, using simple icons, consistent fonts, and easily recognizable buttons.
- **Navigation menus** (e.g., Home, Dashboard, Profile, Contracts, Payments, Help) will be available on all screens.
- All screens will include **standard action buttons** such as Submit, Cancel, Back, and Save.
- The system will support **field tabbing sequences** for ease of form filling.
- **Messages and errors** will be displayed using clear, non-technical language (e.g., “Please upload a valid CNIC in JPG or PDF format.”).
- The layout will support **screen resolutions from 1280x720 and higher**, with responsive behavior for smaller screens.
- The system will aim to follow **accessibility guidelines**, including high-contrast themes and font resizing to support visually impaired users.
- **Shortcut keys** may be added for frequent actions like Ctrl+S to save a form or Esc to close a popup.
- **Help links** will be available on every page, linking to FAQs or support.

6.2 Software interfaces

The SmartRent system will interact with the following external software components:

SI-1: MongoDB Atlas

The system shall use MongoDB Atlas as the cloud-based NoSQL database for storing user profiles, contracts, and transactions.

SI-2: Payment Gateway APIs

The system shall integrate with local payment services (such as JazzCash, Easypaisa, and Bank APIs) for secure rent transactions.

SI-3: Cloud Storage Service

The system shall connect with a cloud storage provider (e.g., AWS S3, Google Cloud) for storing uploaded identity documents and contracts securely.

SI-4: Email Service API

The system shall use a transactional email API (e.g., SendGrid or SMTP server) to send verification, notification, and payment confirmation emails to users.

SI-5: Operating System

The backend server shall run on a Linux-based environment hosted on a cloud platform (e.g., Heroku or AWS EC2).

6.3 Hardware interfaces

The SmartRent platform is a **web-based software system** and does not directly interact with any hardware components. However, the system is expected to run on:

- Standard **desktop or laptop hardware** with internet access.
- **Mobile devices (Android/iOS)** for accessing the web interface.
- Admins may optionally use **scanners or webcams** to verify identity documents, but these are outside the software's control and do not require special integration.

No specific hardware communication protocols are required.

6.4 Communications interfaces

The SmartRent system will use the following communication methods:

CI-1: Web Browser Communication

The system shall be accessible via secure HTTPS connections using modern browsers (Chrome, Firefox, Edge, Safari).

CI-2: Email Notifications

The system shall send emails to users for account verification, document approval status, rent payment confirmations, reminders, and dispute updates.

CI-3: API Communication

Internal modules (e.g., payments, verification, notifications) shall communicate using RESTful APIs.

CI-4: Optional SMS Service (Future Scope)

In future versions, SMS notifications may be integrated for alerts like verification approval or payment success.

7. References

W.K. Chen. *Linear Networks and Systems*. Belmont, CA: Wadsworth, 1993, pp. 123–135.

G. Pevere. “Infrared Nation.” *The International Journal of Infrared Design*, vol. 33, pp. 56–99, Jan. 1979.

D.B. Payne and H.G. Gunhold. “Digital sundials and broadband technology.” *Proc. IOOC-ECOC*, 1986, pp. 557–998.

M. Duncan. “Engineering Concepts on Ice.” Internet: www.iceengg.edu/staff.html, Oct. 25, 2000 [Accessed: Nov. 29, 2003].

MongoDB Atlas Documentation. “Secure Cloud Database Hosting.” Internet: <https://www.mongodb.com/cloud/atlas>, [Accessed: May 30, 2025].

JazzCash Developer Portal. “API Integration Guide.” Internet: <https://developer.jazzcash.com.pk>, [Accessed: May 30, 2025].

SendGrid Documentation. “Transactional Email API.” Internet: <https://docs.sendgrid.com>, [Accessed: May 30, 2025].

Pakistan Electronic Transactions Ordinance. “Digital Signatures and Contracts Law.” Internet: <http://www.moitt.gov.pk>, [Accessed: May 30, 2025].

Figma. “UI Design for Web and Mobile Apps.” Internet: <https://www.figma.com>, [Accessed: May 30, 2025]

Fully Dressed Use Cases

Use Case 1: User Registration and Login

Use Case ID	UC-1
Use Case Name	Register and Login
Actors	Primary: Tenant, Landlord Secondary: None
Description	A user creates an account using email and password, logs into the system, and gains role-based access (Tenant or Landlord).
Trigger	A new user visits the SmartRent platform and clicks on "Sign Up" or "Login".
Preconditions	PRE-1: User has a valid email address. PRE-2: Internet connectivity is available.
Postconditions	POST-1: User account is created or accessed. POST-2: User dashboard is displayed based on role.
Normal Flow	1. User clicks "Register". 2. System displays registration form. 3. User enters email, password, and role (tenant/landlord). 4. System validates email and password format. 5. System creates account and redirects to login. 6. User logs in with credentials. 7. System verifies credentials and displays role-based dashboard.
Alternative Flows	1.1: If email is already used, system shows "Email already registered." 1.2: If password is weak, system prompts password strength requirements. 1.3: If login fails 5 times, account is locked temporarily.
Exceptions	1.0.E1: Server not responding → Show error message. 1.0.E2: Missing required fields → Highlight errors and stop submission.
Business Rules	BR-1: Password must be 8+ characters with a symbol. BR-2: Email must be unique in the system.
Assumptions	Assume most users will use Gmail or Pakistani mobile-linked emails.

Use Case 2: Manual Identity Verification

Use Case ID	UC-2
Use Case Name	Identity Verification by Admin
Actors	Primary: Admin Secondary: Tenant, Landlord
Description	Admin verifies uploaded identity documents submitted by registered users.
Trigger	User uploads ID documents after registration. Admin logs in to verify them.
Preconditions	PRE-1: User account is created and documents are uploaded. PRE-2: Admin is logged in.
Postconditions	POST-1: User status is updated to "Verified", "Rejected", or "Pending". POST-2: Notification sent to the user.
Normal Flow	1. User uploads CNIC or ID proof. 2. System stores the documents securely. 3. Admin views pending verifications. 4. Admin reviews the document. 5. Admin marks status as Verified or Rejected. 6. System sends user a notification.
Alternative Flows	1.1: Admin sends document back for re-upload with comment. 1.2: If document is blurry/invalid, admin marks it as Rejected.
Exceptions	1.0.E1: Admin cannot open document → Show “File corrupted” and flag it. 1.0.E2: Timeout while saving status → Prompt to retry.
Business Rules	BR-1: Verification should complete within 48 hours. BR-2: Only verified users can list or rent properties.
Assumptions	Admin understands government-issued IDs. Manual process will be semi-automated in future.

Use Case 3: Create and Sign Rental Agreement

Use Case ID	UC-3
Use Case Name	Create Digital Rental Agreement
Actors	Primary: Landlord Secondary: Tenant
Description	Landlord creates a rental agreement and shares it with a tenant. Both digitally sign it.
Trigger	Landlord clicks “Create New Contract” in dashboard.
Preconditions	PRE-1: Landlord and tenant accounts are verified. PRE-2: Property is listed and tenant has applied.
Postconditions	POST-1: Blockchain-based contract is created and stored. POST-2: Notification sent to tenant and landlord.
Normal Flow	1. Landlord selects a tenant and clicks “Create Agreement”. 2. System displays form with rent, duration, rules, etc. 3. Landlord fills the form and previews the contract. 4. System validates fields and saves to blockchain. 5. Tenant receives a notification. 6. Tenant reviews and signs the contract. 7. Contract is finalized and saved permanently.
Alternative Flows	1.1: Tenant requests changes → System reopens the draft for landlord editing. 1.2: Tenant rejects → Contract status updated to “Declined”.
Exceptions	1.0.E1: Blockchain not accessible → Show “Temporary server error.” 1.0.E2: Required fields missing → Highlight and stop submission.
Business Rules	BR-1: No contract can be edited once both parties have signed. BR-2: Contract must comply with Pakistan Rental Laws.
Assumptions	Assume contract templates comply with legal clauses. System has access to blockchain service.

Use Case 4: Online Rent Payment

Use Case ID	UC-4
Use Case Name	Pay Monthly Rent
Actors	Primary: Tenant Secondary: Landlord, Payment Gateway
Description	A tenant pays monthly rent through an integrated payment gateway. Confirmation is sent to both tenant and landlord.
Trigger	Tenant logs in and navigates to the "Pay Rent" page.
Preconditions	PRE-1: A signed rental contract must exist. PRE-2: Tenant is logged in and verified. PRE-3: Payment gateway is online.
Postconditions	POST-1: Payment record is created. POST-2: Receipts are sent to both tenant and landlord. POST-3: Payment history updated.
Normal Flow	1. Tenant clicks “Pay Rent” for the active property. 2. System displays rent amount and due date. 3. Tenant selects payment method (e.g., JazzCash, bank transfer). 4. Tenant enters payment details. 5. System connects with payment API. 6. On success, receipt is generated. 7. Email/SMS notifications sent to both parties.
Alternative Flows	1.1: Tenant enables auto-pay → System schedules monthly payment automatically. 1.2: Partial payment allowed → System marks balance due and sets reminder.
Exceptions	1.0.E1: Payment fails due to gateway timeout → Display “Transaction failed. Try again.” 1.0.E2: Invalid account/card details → Highlight and prompt re-entry.
Business Rules	BR-1: Payments must be recorded in PKR only. BR-2: Rent must be paid by the 5th of each month.
Assumptions	Assume payment gateways are licensed in Pakistan and support callback confirmations.

Use Case 5: Leave Review After Tenancy

Use Case ID	UC-5
Use Case Name	Submit Tenant or Landlord Review
Actors	Primary: Tenant, Landlord
Description	After contract completion, both parties can review each other using a rating system and optional comments/photos.
Trigger	A rental contract is marked as completed.
Preconditions	PRE-1: Rental period is complete. PRE-2: User is logged in and verified.
Postconditions	POST-1: Review is published on profile. POST-2: Trust score is recalculated.
Normal Flow	1. System prompts user to leave a review. 2. User gives star rating for parameters (e.g., communication, payment, cleanliness). 3. Optional comment and photo/video evidence are added. 4. System validates input and saves review. 5. Reviewed user gets a notification.
Alternative Flows	1.1: If user disputes review → Admin receives review flag and decides. 1.2: User edits review within 24 hours.
Exceptions	1.0.E1: User tries to review without contract → Show “Review not allowed.”
Business Rules	BR-1: Review can be submitted once per contract. BR-2: Review must be moderated if flagged.
Assumptions	Assume tenants and landlords value trust scores for future agreements.

Use Case 6: Dispute Resolution Submission

Use Case ID	UC-6
Use Case Name	Submit Rental Dispute
Actors	Primary: Tenant, Landlord Secondary: Admin
Description	A user files a dispute about payments, property conditions, or contract violations. Admin reviews and resolves the case.
Trigger	Tenant or landlord clicks “Report a Problem” from the contract or payment history page.
Preconditions	PRE-1: User is logged in and linked to an active or recently completed contract. PRE-2: Dispute reason and evidence are provided.
Postconditions	POST-1: Dispute is added to the admin dashboard. POST-2: Admin responds with decision. POST-3: System logs all actions for audit.
Normal Flow	1. User clicks “Submit Dispute”. 2. System shows structured form (reason, description, attachments). 3. User uploads evidence (PDF, JPG). 4. System validates file types/sizes. 5. Admin views case in dashboard. 6. Admin responds within 72 hours. 7. System sends resolution notification to both parties.
Alternative Flows	1.1: Admin requests additional evidence → User reopens and adds more info. 1.2: Dispute escalated to third-party arbitrator (future scope).
Exceptions	1.0.E1: File upload fails → Prompt retry or allow email submission. 1.0.E2: Admin misses 72-hour SLA → Escalation alert triggered.
Business Rules	BR-1: Only verified users may submit disputes. BR-2: Disputes must be raised within 30 days of contract issue.
Assumptions	Assume evidence (e.g., payment receipts or chat logs) is authentic.

Use Case 7: AI-Based Rent Price Suggestion

Use Case ID	UC-7
Use Case Name	Get AI Rent Suggestion
Actors	Primary: Landlord
Description	A landlord receives an AI-generated rent suggestion based on market data, location, and property attributes.
Trigger	Landlord clicks “Get Price Suggestion” while creating or editing a rental listing.
Preconditions	PRE-1: Landlord is verified and logged in. PRE-2: Property location, size, and type are filled.
Postconditions	POST-1: Suggested rent range is displayed with comparison chart. POST-2: Landlord may accept or override the suggestion.
Normal Flow	1. Landlord enters property details. 2. System sends details to AI model. 3. AI returns a suggested rent range. 4. System shows average area prices, historical trends, and factors considered. 5. Landlord reviews suggestion and clicks “Accept” or “Override”.
Alternative Flows	1.1: If landlord overrides, system logs reason for future model improvement.
Exceptions	1.0.E1: Market data unavailable → Show “No suggestion available for this area.”
Business Rules	BR-1: AI model must be trained on at least 6 months of real estate data. BR-2: Suggestions must show transparency (influencing features).
Assumptions	Assume property data and external market data APIs are accurate.

Use Case 8: AI Budget-Based Location Suggestion

Use Case ID	UC-8
Use Case Name	Get Area Suggestions Based on Budget
Actors	Primary: Tenant
Description	A tenant enters a rental budget, and the AI suggests affordable and safe neighborhoods with listings.
Trigger	Tenant clicks “Find Areas Within My Budget”.
Preconditions	PRE-1: Tenant is logged in. PRE-2: Budget and preferred city are provided.
Postconditions	POST-1: System displays recommended areas with matching listings. POST-2: User may save or share results.
Normal Flow	1. Tenant enters budget and city. 2. System retrieves nearby listings. 3. AI filters locations based on safety score, price trends, and commute distance. 4. System shows top 3–5 matching neighborhoods. 5. Tenant clicks on a location to explore listings.
Alternative Flows	1.1: If no exact match → System suggests alternatives with a 10–15% budget margin.
Exceptions	1.0.E1: Insufficient data for city → Show “No data available. Try a nearby area.”
Business Rules	BR-1: Only show areas with 80%+ listing coverage. BR-2: Display must include commute time and crime/safety score.
Assumptions	Assume tenant is open to alternate areas in the same city. AI model uses updated listing and safety data.

Use Case 9: Manage Rental Documents

Use Case ID	UC-9
Use Case Name	Upload/View Rental Documents
Actors	Primary: Tenant, Landlord
Description	Users upload or view rental documents (like CNIC, ownership proofs, utility bills) stored securely in the system.
Trigger	User clicks “Upload Documents” or opens the document section in a contract.
Preconditions	PRE-1: User is logged in and verified. PRE-2: Document is in accepted format (PDF, JPG, PNG).
Postconditions	POST-1: Document is stored securely in encrypted storage. POST-2: Document is view-only for the counterparty.
Normal Flow	1. User clicks “Upload Document”. 2. System shows file type and size limits. 3. User selects file. 4. System encrypts and uploads it. 5. File appears in the relevant contract or profile tab. 6. Counterparty sees “View” access but no download/delete options.
Alternative Flows	1.1: Admin uploads compliance document → All parties receive access and notification.
Exceptions	1.0.E1: File too large → Prompt user to compress file and retry. 1.0.E2: Unauthorized access attempt → System logs and alerts admin.
Business Rules	BR-1: Maximum file size is 10 MB. BR-2: Only verified users can upload documents. BR-3: No deletion allowed after contract is signed.
Assumptions	Assume users upload only authentic and relevant documents. Storage is cloud-based and encrypted.